

EXPERIMENT-7

Implement Logistic Regression algorithm.

Code:

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

print("="*70)
print("      LOGISTIC REGRESSION")
print("="*70)

uploaded=files.upload()

data=pd.read_csv("data.csv")

print("\nDataset")
print(data.head())

data=data.drop(["id","Unnamed: 32"],axis=1)

X=data.drop("diagnosis",axis=1)

y=data["diagnosis"]
```

```

encoder=LabelEncoder()

y=encoder.fit_transform(y)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.3,random_state=42)
model=LogisticRegression(max_iter=5000)
print("\nTraining Logistic Regression Model...")
model.fit(X_train,y_train)
prediction=model.predict(X_test)
print("\nActual Values")
print(y_test)
print("\nPredicted Values")
print(prediction)
print("\nAccuracy")
print(round(accuracy_score(y_test,prediction)*100,2),"%")
print("\nConfusion Matrix")
print(confusion_matrix(y_test,prediction))
sample=X.iloc[0].values
result=model.predict(sample)
print("\nPrediction for First Record")
if result[0]==1:
    print("Malignant")
else:
    print("Benign")

```

Sample Input:

```

...  =====
      LOGISTIC REGRESSION
      =====
      Choose Files data.csv
      data.csv(text/csv) - 125204 bytes, last modified: 8/5/2026 - 100% done
      Saving data.csv to data (1).csv

```

Sample Output:

... Dataset

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

	... texture_worst	perimeter_worst	area_worst	smoothness_worst	\
0	... 17.33	184.60	2019.0	0.1622	
1	... 23.41	158.80	1956.0	0.1238	
2	... 25.53	152.50	1709.0	0.1444	
3	... 26.50	98.87	567.7	0.2098	
4	... 16.67	152.20	1575.0	0.1374	

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

Training Logistic Regression Model...

Actual Values

```
[0 1 1 0 0 1 1 1 0 0 0 1 0 1 0 1 0 0 0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1
 0 1 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0 0 1 1 0 1
 0 0 0 1 0 0 1 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 0 1 1 0 1 1 0 0 0 1 0 0 1
 0 0 1 0 1 0 0 0 1 0 0 0 1 0 1 1 0 0 1 1 1 0 0 0 1 0 0 0 1 0 1 0 0 1 0 1 1
 1 0 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0]
```

Predicted Values

```
[0 1 1 0 0 1 1 1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1
 0 1 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 0 0 0 1 1 0 0 1 1 0 1
 0 0 0 1 0 0 1 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 0 1 1 0 1 1 0 0 0 1 0 0 1
 0 1 1 0 1 0 0 0 1 1 0 0 1 0 1 1 0 0 1 1 1 0 0 0 1 0 0 0 1 0 1 0 0 1 0 1 1
 1 0 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0]
```

Accuracy

97.66 %

Confusion Matrix

```
[[106  2]
 [ 2 61]]
```

Prediction for First Record

Malignant